

Boas Práticas de Programação: Parte 1

Por: Prof. Dr. Debora B. Aires



Código Limpo

- Refere-se a escrever código que é fácil de entender, manter e estender. Envolve aderir a boas práticas de programação, como seguir convenções de nomenclatura, manter funções curtas e focadas, evitar duplicação, comentar apenas quando necessário e aplicar princípios de design, como os mencionados no SOLID.



Refatoração

- A refatoração envolve a reestruturação do código-fonte existente para melhorar sua legibilidade, manutenibilidade e eficiência, sem alterar o comportamento externo do software. É um processo contínuo que visa eliminar a duplicação, melhorar a estrutura do código e aplicar os princípios de design para criar um código mais limpo e compreensível.
 - Refatorações servem para melhorar o design do código, e não o funcionamento do sistema. Uma refatoração não deve influenciar em nada no funcionamento.
- 

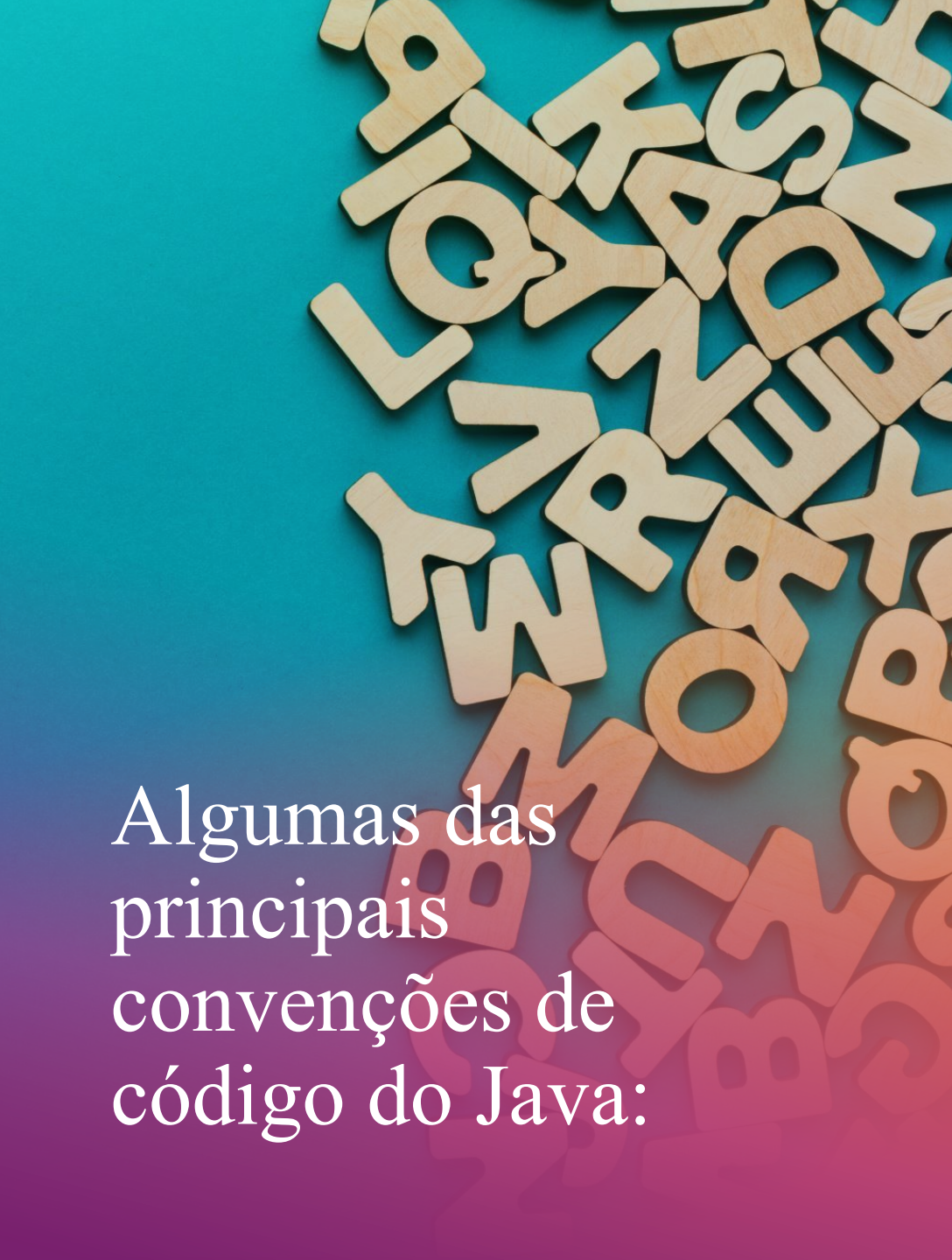
Convenção da linguagem Java

- Classe -> iniciando com letra maiúscula;
 - Use PascalCase para palavras compostas;
- Variáveis, métodos, atributos -> iniciando com letra minúscula;
 - Use camelCase para palavras compostas;

PascalCase

Convenções influenciam na
legibilidade do código



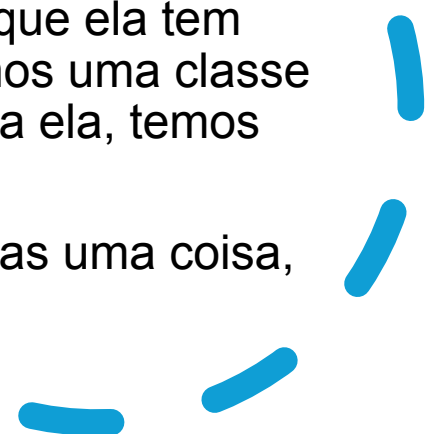


Algumas das principais convenções de código do Java:

- Nomes de classes devem começar com letra maiúscula e usar a convenção PascalCase (também conhecida como Upper CamelCase).
 - Exemplo: MinhaClasse.
- Nomes de métodos devem começar com letra minúscula e usar a convenção camelCase.
 - Exemplo: meuMetodo().
- Nomes de constantes devem ser totalmente em letras maiúsculas, separadas por underline.
 - Exemplo: MINHA_CONSTANTE.
- Nomes de variáveis devem começar com letra minúscula e usar a convenção camelCase.
 - Exemplo: minhaVariavel.
- Nome de package tudo minúsculo
 - Exemplo: exercicio
- Todas as linhas de código devem ter no máximo 80 caracteres de largura para facilitar a leitura.
- Recomenda-se usar espaços em branco para separar operadores, palavras-chave e elementos de controle de fluxo.
 - Exemplo: if (condicao) {.
- Use comentários para documentar seu código, explicando o que ele faz e por que ele faz isso. Comentários devem ser claros e concisos.
 - Esse ponto anterior, referente aos comentários, é polêmico, pois muitas pessoas desenvolvedoras consideram que um bom código deve ser autoexplicativo. Se você utiliza nomes intuitivos e descritivos para suas variáveis e métodos, fica mais claro de entender o que está acontecendo no código, dispensando a necessidade de uso de comentários.

“Classes não coesas tendem a crescer indefinidamente, o que as tornam difíceis de manter”

Coesão

- Quando temos uma classe elaborada de forma que tenha um único e bem focado propósito, dizemos que ela tem uma alta coesão, e isso é bom. Quando temos uma classe com propósitos que não pertencem apenas a ela, temos uma baixa coesão, o que é ruim.
 - Cada classe deve ser responsável por apenas uma coisa, e deve executar esta tarefa muito bem
- 

Coesão

Refatorando...

```
public class Funcionario {  
    private String nome;  
    private String cpf;  
    private Cargo cargo;  
    private BigDecimal salario;  
    private String cep;  
    private String logradouro;  
    private String bairro;  
    private String cidade;  
    private String uf;  
  
    public void formatarCpf() {  
        // logica para formatacao do CPF  
    }  
  
    public void formatarCep() {  
        // logica para formatacao do CEP  
    }  
  
    public void completarEndereco() {  
        // logica para consulta ao WS dos correios  
    }  
}
```



```
public class Funcionario {  
  
    private String nome;  
    private String cpf;  
    private Cargo cargo;  
    private BigDecimal salario;  
  
    public boolean isGerente() {  
        return Cargo.GERENTE == this.cargo  
    }  
}
```

Encapsulamento

“Classes não encapsuladas permitem violação de regras de negócio, além de aumentarem o acoplamento”

encapsulamento é uma forma de manter os objetos das nossas classes protegidos, fornecendo apenas o que é estritamente necessário para o mundo exterior.

O fato de criar getters e setters para tudo, na verdade, quebra o encapsulamento da nossa classe.

Ao encapsular o acesso a determinados dados, liberando acesso apenas ao necessário, os objetos da nossa classe se tornam mais fáceis de serem utilizados.

Encapsulamento

Refatorando...

```
public class Funcionario {  
  
    private String nome;  
    private String cpf;  
    private Cargo cargo;  
  
    // double apenas para fins didaticos  
    private double salario;  
  
    public void setSalario(double salario) {  
        this.salario = salario;  
    }  
}
```



```
Funcionario novo = carregarDoBancoDeDados();  
novo.setSalario(100000);
```

```
public class Funcionario {  
  
    private String nome;  
    private String cpf;  
    private Cargo cargo;  
  
    // double apenas para fins didaticos  
    private double salario;  
  
    public void reajustarSalario(double aumento) {  
        double percentualReajuste = (aumento / salario) * 100;  
  
        if (percentualReajuste > 40) {  
            throw new IllegalArgumentException(  
                "percentual de reajuste deve ser inferior a 40%  
            );  
        }  
  
        this.salario += aumento;  
    }  
}
```

Acoplamento

- É o grau em que uma classe conhece a outra. Se o conhecimento da classe A sobre a classe B for através de sua interface, temos um baixo acoplamento, e isso é bom. Por outro lado, se a classe A depende de membros da classe B que não fazem parte da interface de B, então temos um alto acoplamento, o que é ruim.
- É fato que, se estamos organizando o nosso código, seguindo as recomendações da orientação a objetos, algum acoplamento acontecerá. Algumas classes precisarão de outras, para que não tenham muitas responsabilidades. Cabe a nós medir quando faz sentido adicionar tal acoplamento com as dependências

“Classes acopladas causam fragilidade no código da aplicação, o que dificulta sua manutenção”

Acoplamento

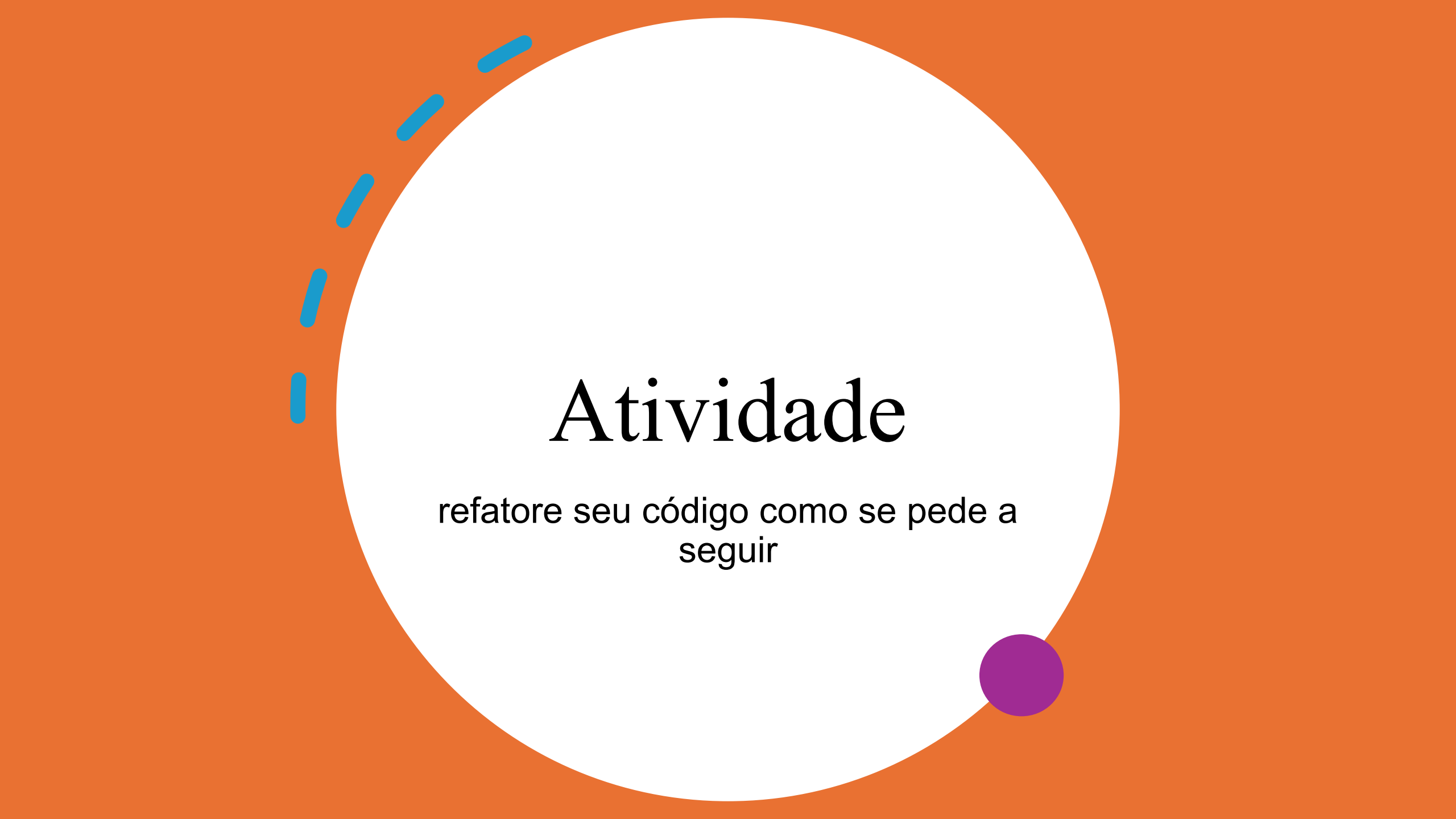
Refatorando...

```
Funcionario funcionario = carregarDoBancoDeDados();  
  
double valorTotalReajustes = 0;  
List<Reajuste> reajustes = funcionario.getReajustes();  
for (Reajuste r : reajustes) {  
    valorTotalReajustes += r.getValor();  
}
```



```
Funcionario funcionario = carregarDoBancoDeDados();  
double reajustes = funcionario.getValorTotalRecebidoEmReajustes();
```





Atividade

refatore seu código como se pede a seguir



Não use
variáveis
globais!

- Variáveis globais devem ser evitadas, pois elas dão origem a um tipo de **acoplamento ruim**. Por isso, elas também constituem um code smell. O principal motivo é que variáveis globais dificultam o entendimento de um módulo de forma independente dos demais módulos de um sistema.
- Em linguagens como Java, atributos estáticos de classes funcionam exatamente como variáveis globais. Logo, eles também representam um code smell.

• <https://engsoftmoderna.info/cap9.html#vari%C3%A1veis-globais>

Não use variáveis globais!

- As variáveis globais são um **code smell** porque criam **acoplamento global**, geram **efeitos colaterais difíceis de rastrear** e dificultam **testes unitários**.
- **Problemas das Variáveis Globais**
- **Acoplamento oculto**: Qualquer função pode alterar o valor da variável sem aviso prévio.
- **Falta de previsibilidade**: O estado do programa depende da ordem de execução das funções.
- **Concorrência perigosa**: Várias partes do sistema tentando mudar o mesmo dado ao mesmo tempo causam falhas.
- **Testes difíceis**: Fica complexo testar uma função isolada se ela depende de um estado global mutável.

Boas Práticas para Substituição:

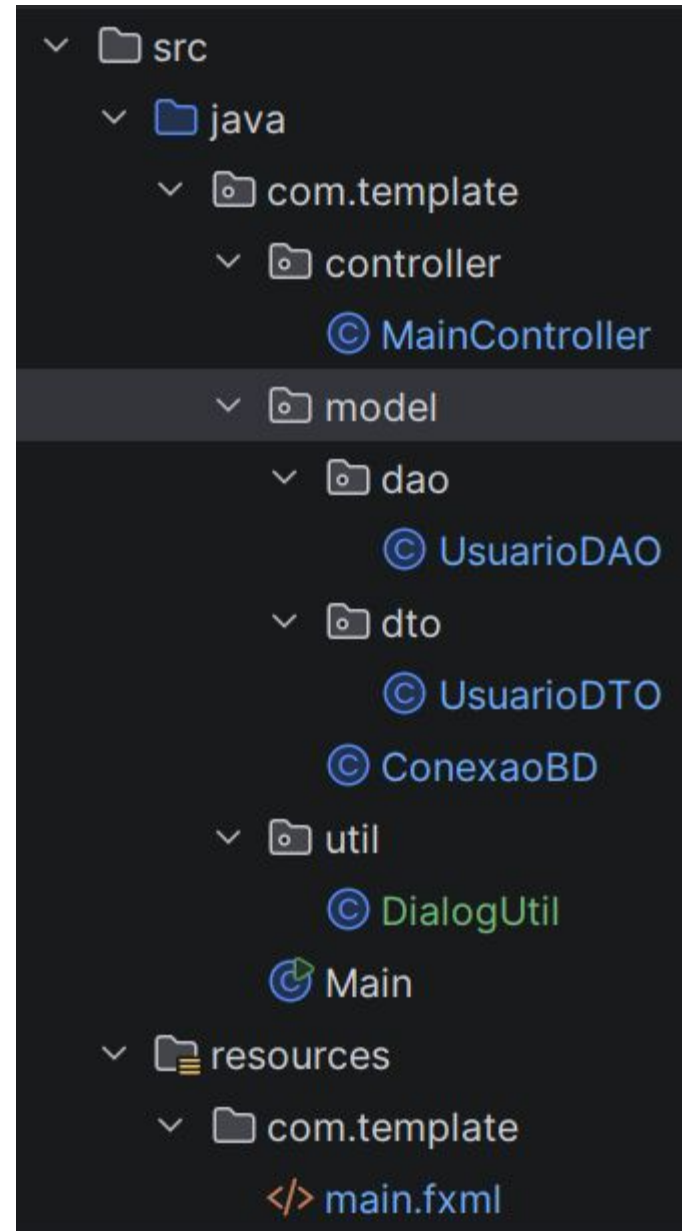
- **Injeção de dependência**: Passar os dados necessários explicitamente como parâmetros para funções ou classes.
- **Escopo local**: Manter as variáveis restritas apenas às funções ou blocos onde elas realmente são usadas.
- **Encapsulamento**: Usar objetos ou módulos para controlar o acesso e a modificação dos dados de forma segura.

Dê nomes autoexplicativos para seus métodos e atributos

Use camelCase para variáveis, atributos e métodos e PascalCase para classes;

- Exemplo: não use `handleButtonAction`, use `btnCadastrarAction` por exemplo;
- Use `btn` pra botão, `lbl` para label e `txt` para `TextField`, `tbl` para `TableView`;
- Dê um nome adequado para o projeto: `AtividadeMVC`, por exemplo;

Organize seu projeto para o padrão MVC



Criar um DialogUtil

- Não use o console para dar alertas!
- Customise os alertas para cada situação e os utilize no controller.

```
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;

public class DialogUtil {
    public static void showError(String mensagem) {
        Alert alert = new Alert(AlertType.ERROR);
        alert.setTitle("Erro");
        alert.setHeaderText(null);
        alert.setContentText(mensagem);
        alert.showAndWait();
    }

    public static void showInfo(String mensagem) {
        Alert alert = new Alert(AlertType.INFORMATION);
        alert.setTitle("Informação");
        alert.setHeaderText(null);
        alert.setContentText(mensagem);
        alert.showAndWait();
    }
}
```

Use o Alert (DialogUtil) em determinadas situações

Para informar erros;

Para informar sucesso;

Para confirmar se o usuário
realmente deseja deletar/atualizar um
dado;

NÃO USE O CONSOLE!

Atualize o catch de cada try



```
public class UsuarioDAO {  
    private static final Logger logger = Logger.getLogger(UsuarioDAO.class.getName());  
    ArrayList<UsuarioDTO> listaUsuarios = new ArrayList<>();  
}
```

```
} catch (SQLException e) {  
    logger.log(Level.SEVERE, "Erro ao listar usuário", e);  
    showError("Erro ao ao listar usuário.");  
}
```